Building the Transformer Text Generation and Sampling Laboratory

A step-by-step build and usage guide.

This document explains, step by step, how to build, run and extend the Transformer Text Generation and Sampling Laboratory. It also lists every command the CLI supports, with copy-paste-ready examples.

Reading order: Part 1 sets up the project, Part 2 runs the first experiments, Part 3 walks through each CLI command, Part 4 covers the optional charts and neural backends, and Part 5 explains how this PDF is generated.

Part 1 - Build it from scratch, step by step

Step 1. Install Python

Requirement: Python 3.9 or newer.

- Verify your installation:

```
python --version
```

- If "python" is not recognised, install Python from python.org and make sure "Add Python to PATH" is ticked during setup.
- On some systems the launcher is called "py" instead of "python".

Step 2. Get the project

Clone the repository and enter the folder:

```
git clone https://github.com/dhonith12/Transformer-Text-Generation-and-Sampling-Laboratory.git
cd Transformer-Text-Generation-and-Sampling-Laboratory
```

If you already downloaded a ZIP, simply unzip it and open a terminal inside the resulting folder. The folder must contain run_lab.py and the lab/ directory at the same level.

Step 3. Create a virtual environment

A virtual environment keeps this project's packages separate from the rest of your system.

```
python -m venv .venv
```

Step 4. Activate the environment

Windows (PowerShell):

```
.venv\Scripts\Activate.ps1
```

Windows (cmd.exe):

```
.venv\Scripts\activate.bat
```

macOS / Linux:

```
source .venv/bin/activate
```

After activation your prompt usually shows "(.venv)". You can confirm the interpreter being used with:

```
python -c "import sys; print(sys.executable)"
```

Step 5. (Optional) install extra packages

The core laboratory uses only the Python standard library, so this step is optional.

```
# charts and numeric helpers
pip install -r requirements.txt
```

- numpy>=1.24 numeric helpers used by the metrics / report code
- matplotlib>=3.7 charts written to results/*.png

```
# neural backends (HuggingFace GPT-2 and the trainable tiny GPT)
pip install torch transformers
```

Step 6. Verify the install

Run the dependency-free n-gram backend on a short prompt:

```
python run_lab.py compare --backend ngram --prompt "Once upon a time" --tokens 60
```

If you see a table of generations and a metrics table, the build is complete.

Troubleshooting quick reference

- "'python' is not recognized" -> Python is not on PATH.
- "Script not found / runlab.py" -> the driver is run_lab.py (underscore, not "runlab").
- "ModuleNotFoundError: lab" -> run from the project root.
- A command that seems to hang -> the "auto" backend chose HuggingFace GPT-2; add "--backend ngram".
- "invalid choice" for --strategy -> valid names are greedy, random, temperature, topk, topp, topk_topp, min_p. Note the underscore in topk_topp.
- "ModuleNotFoundError: matplotlib" -> pip install matplotlib, or add --no-plots.

Part 2 - Run your first experiment

The offline n-gram model needs no PyTorch and no internet download:

```
python run_lab.py compare --backend ngram --prompt "Once upon a time" --tokens 60
```

This compares decoding strategies such as greedy, temperature, top-k and top-p side by side, printing both the generated text and a rubric of metrics (diversity, repetition, randomness, coherence, relevance, factuality, safety, helpfulness).

Part 3 - Command reference, one step at a time

Every command below is verified against the current code. Adding "--backend ngram" keeps runs instant and fully offline.

3.1 compare - side-by-side decoding strategies

```
python run_lab.py compare --backend ngram --prompt "Once upon a time" --tokens 60
```

What it does: generates from the same prompt with ten standard decoding configurations (greedy, random, two temperatures, two top-k settings, two top-p settings, top-k+top-p, min-p) and scores each one.

Key flags: --prompt, --tokens, --seed, --no-json, --no-plots.

3.2 sweep - explore one parameter

```
python run_lab.py sweep --backend ngram --param temperature --values
0.1,0.3,0.6,0.9,1.2,1.5,2.0
```

What it does: holds everything fixed and varies a single knob, printing a metrics table so you can watch the diversity / coherence trade-off.

Allowed --param values: temperature, top_k, top_p, min_p, repetition_penalty.

3.3 analyze - deep dive into one generation

```
python run_lab.py analyze --backend ngram --strategy topp --top-p 0.9
```

What it does: produces a detailed metric table for a single configuration, including perplexity, continuity, entropy traces and safety flags.

Key flags: --strategy, --top-p, --top-k, --min-p, --temperature, --tokens.

3.4 safety - controlled red-team suite

```
python run_lab.py safety --backend ngram --strategy topk_topp
```

What it does: runs the built-in test cases (hallucination, misinformation, bias, unsafe, prompt injection, plus benign controls), comparing unguarded behaviour against the SafetyGuard defence, and prints pass rates per category.

Note the strategy name: topk_topp (with an underscore).

3.5 compute - test-time compute experiments

```
python run_lab.py compute --backend ngram --n-values 1,2,4,8,16 --vote "What is 2+2?"
```

What it does: three experiments in one run -

- best-of-N reranking (quality as a function of N),
- token-budget curves (quality vs generated length),
- self-consistency majority voting over N samples.

Key flags: --n-values, --budgets, --vote, --vote-n.

3.6 report - the full laboratory report

```
python run_lab.py report --backend ngram --no-plots
```

What it does: runs the comparison, the temperature sweep, the test-time compute experiments and the safety suite, then writes a complete markdown report to results/report.md, with all raw data in results/*.json.

Drop --no-plots (after installing matplotlib) to also embed PNG charts:

```
python run_lab.py report --backend ngram
```

3.7 chat - interactive exploration

```
python run_lab.py chat --backend ngram
```

What it does: opens an interactive loop. Type a prompt to generate, or type a strategy name (greedy, topp, temperature, ...) to switch decoding on the fly. Type "quit" to exit. Live metrics are printed after each generation.

3.8 train - the optional tiny transformer

```
pip install torch
```

```
python run_lab.py train --steps 300
```

What it does: trains a miniature GPT from scratch on the bundled corpus and saves weights to artifacts/tiny_gpt.pt.

Then use your trained model through the same decoding code:

```
python run_lab.py compare --backend tiny --checkpoint artifacts/tiny_gpt.pt
```

Key flags: --steps, --batch-size, --lr, --block-size, --n-embd, --n-head, --n-layer, --corpus, --checkpoint.

Part 4 - Optional: charts and neural backends

4.1 Charts

```
pip install numpy matplotlib
```

```
python run_lab.py report
```

Without --no-plots, commands write PNG charts to results/ alongside the JSON data.

4.2 HuggingFace GPT-2

```
pip install torch transformers
```

```
python run_lab.py compare --backend hf --model gpt2
```

The first run downloads the GPT-2 weights (requires internet). Subsequent runs use the local cache. Because this is slower than the n-gram model, use it when you specifically want a "real" transformer.

4.3 The tiny trainable GPT

```
pip install torch
```

```
python run_lab.py train --steps 300
```

```
python run_lab.py compare --backend tiny --checkpoint artifacts/tiny_gpt.pt
```

4.4 Backend auto-selection

The default --backend auto tries hf -> tiny -> ngram, falling back when dependencies or checkpoints are missing. If you want speed and full offline operation, pass --backend ngram explicitly.

Part 5 - Where results live, and regenerating this PDF

Result locations

- JSON experiment outputs : results/*.json

- Charts : results/*.png
- Full report : results/report.md
- Tiny GPT weights : artifacts/tiny_gpt.pt

Regenerating this document

This PDF is generated from a plain-text source by a small standard-library-only script, so it can always be rebuilt:

```
python docs/build_pdf.py
```

Inputs / outputs:

```
docs/Building_the_LLM_Lab.txt  -- human-editable source text
docs/build_pdf.py              -- generator (no third-party packages)
docs/Building_the_LLM_Lab.pdf  -- the file you are reading
```

To change the guide, edit the .txt file and re-run the generator.

Part 6 - Extending the laboratory

Ideas and entry points:

- New decoding strategies -> lab/decoding.py (register in STRATEGIES)
- New metrics -> lab/metrics.py
- New safety test cases -> lab/safety.py
- New backends -> lab/backend.py
- New visualizations -> lab/visualize.py

Keep implementations transparent and dependency-free where possible, add docstrings, and keep the code readable for students.

Built for learning. Designed to be transparent. Run anywhere with Python.